Marksheet✓

Project Explained

AI answer-sheet evaluation: reads handwritten exam scripts, scores each answer against a rubric by meaning, and flags the answers a human should double-check.

Built for: CNV AI Labs, "AI/ML Engineer: Answer Sheet Evaluation" timed assignment · Codebase: ~2,300 lines, 30 source files · Python (FastAPI) + TypeScript (React) + SQLite + Google Gemini API

---

1 Project overview

2 End-to-end flow

3 Flowcharts (workflow + sub-flows)

4 System architecture

5 Technologies used

6 Important code mapping

7 Interview explanation (2–3 min)

8 Deep technical explanation

9 Interview questions & answers (50)

10 One-page cheat sheet

A Appendix: Mermaid sources

> **How this document was produced.** Everything below is derived from the actual files in `Ai_Project_Test/`. Anything the codebase does *not* contain (authentication, message queues, cloud hosting, monitoring, automated tests, CI/CD, Docker) is marked NOT PRESENT rather than assumed.

1 Project overview

What the project does

Marksheet is a web application (plus a command-line tool) that takes **scanned or photographed handwritten answer sheets** and a **rubric** (answer key split into mark-bearing points). It then:

1. **Extracts** the handwriting into text using a vision AI model, reading every page *twice* with two different models;
2. **Structures** the text by question number, joining answers that continue onto the next page or are interrupted by a diagram;
3. **Scores** each answer against each rubric point *by meaning* (not keyword matching), three times, and takes the median;
4. **Flags confidence** (high / medium / low) with a one-line reason, so a human only re-checks the uncertain answers;
5. Shows results in a browser UI and **exports JSON / CSV**.

What problem it solves, and why it is needed

From the assignment brief (`AIML Engineer - Answer Sheet Evaluation - 120min Assignment-1.pdf`): about 250 million students in India sit exams every year and are graded by hand; two evaluators can differ by 15–20% on the same paper. Manual grading is slow, inconsistent and hard to audit. The project is a scaled-down slice of an AI

pipeline that makes grading **consistent** (same rubric, same rules, every time), **explainable** (each mark has a verdict and a quote as evidence) and **safe** (it says when it is unsure instead of guessing silently).

ASPECT	IN THIS PROJECT
Main input	1–30 page images (JPG/PNG/WEBP/BMP/TIFF) or PDFs of one handwritten answer script, plus a rubric chosen from the database (seeded from `data/rubrics/python_rubric.json`: 8 Python questions, 37 marks).
Main processing	Image cleanup (OpenCV) → two independent Gemini vision transcriptions per page → question segmentation state machine → rubric-criterion LLM grading ×3 with median → embedding similarity sanity check → rule-based confidence.
Main output	Per question: extracted text, pages, score, score vs. key (e.g. `2.5/4 · c1✓ c2✓ c3½ c4✗`), per-criterion verdict + evidence, confidence label + reason, all signals. Stored in SQLite; shown in the UI; exported as `results.json` / `results.csv`.

My role / contribution

> **Not identifiable from the code itself:** there is no git history, no author field and no commit log in the project folder.
>
> **What the project files do show:**
>
> - `data/SOURCE.md` states the sample answer sheet was handwritten by the candidate and photographed.
> - `data/QUESTION_PAPER.md` and `data/rubrics/python_rubric.json` define the questions and answer key for this project.
> - `APPROACH.md` is the approach note the brief asks for.
>
> **How to describe it honestly:** the brief explicitly allows any AI tool ("use them as you would on the job"). If AI coding assistance was used, say so. Your contribution is:
>
> - framing the problem and choosing the stack (Gemini, FastAPI, React, SQLite);
> - creating the test data and rubric;
> - directing, reviewing and verifying the implementation;
> - understanding every tradeoff, which is exactly what this document prepares you to defend.

Measured result on the sample sheet (from `outputs/results.json`)

Q	SCORE	CONFIDENCE	REASON GIVEN BY THE PIPELINE
Q1	2.5 / 4	medium	Grading runs varied by 0.5 marks
Q2	1 / 5	**low**	Repeated grading runs disagree by 1 mark
Q3	1 / 5	high	Readers agree, grading runs consistent
Q4	1.5 / 4	medium	Grading runs varied by 0.5 marks
Q5	0 / 5	high	Question number written but no answer given, 0 awarded
Q6	0 / 3	high	Readers agree, grading runs consistent
Q7	0.5 / 6	medium	Grading runs varied by 0.5 marks
Q8	2 / 5	high	Readers agree, grading runs consistent

Total 8.5 / 37 · the two OCR readers agreed 98.7–100% on every answer · re-run from cache: 0.8 s

2 End-to-end flow

The web path is traced below; the CLI path (`backend/cli.py`) calls the same `evaluate()` function directly and skips the API, background job and database.

```
User (browser)
 → React UI (Home.tsx, DropZone.tsx)                choose pages + rubric
 → POST /api/evaluations (routes.py)                validate, store row, start job
 → Background job (jobs.py)                         preprocess + save page images
 → Pipeline runner (runner.py)
     → Extract   (extract.py → gemini.py → Google Gemini vision)   [cache.db]
     → Structure (segment.py)                       state machine by question
     → Score     (score.py → gemini.py → Gemini judge ×3 + embeddings)   [cache.db]
     → Confidence (confidence.py)                   rules → high / medium / low
 → SQLite app.db (db.py)                            result JSON persisted
 → React UI polls GET /api/evaluations/{id} (EvaluationView.tsx) → renders cards
 → GET /api/evaluations/{id}/export?fmt=csv|json (export.py) → download
```

1 User selects pages and a rubric

WHAT	User drags images/PDFs onto the drop zone, reorders them (‹ ›), picks a rubric, clicks Evaluate. Files are filtered by extension and naturally sorted (page2 before page10).
WHERE	`frontend/src/pages/Home.tsx` → `Home()`, `submit()`; `components/DropZone.tsx` → `DropZone()`, `add()`, `move()`; `api.ts` → `api.evaluate()`
IN → OUT	Browser `File` objects + rubric id → `multipart/form-data` POST (`rubric_id`, `files[]`)
WHY	Page order matters: an answer continuing on the next page can only be stitched correctly if pages are in order.

2 API validates and creates an evaluation

WHAT	Checks the rubric exists (404), 1–30 files (400), allowed extension (400), each file ≤ 25 MB (400). Creates an `EvaluationRow` (status `queued`) with a 12-hex-char id, starts the background task, returns immediately.
WHERE	`backend/app/api/routes.py` → `create_evaluation()`; `db.py` → `get_rubric()`, `EvaluationRow`; `api/jobs.py` → `start()`
IN → OUT	Form data → `{"id": "574a1c9fbc3e"}`
WHY	A full evaluation takes 20–60 s; returning at once keeps the HTTP request short and lets the UI show live progress.

3 Background job: preprocess pages

WHAT	PDF pages rendered at 200 dpi (PyMuPDF); photos rotated per EXIF; resized to ≤ 2000 px; grayscale; deskew by projection profile (tests −6°…+6° in 0.5° steps, picks the angle with the sharpest row-sum variance); CLAHE contrast; JPEG q88; SHA-256 hash per page. Page images saved to `storage/evals/<id>/pageN.jpg`.
WHERE	`api/jobs.py` → `_run()`; `pipeline/preprocess.py` → `prepare()`, `_load()`, `_clean()`, `_estimate_skew()`
IN → OUT	`(filename, bytes)` list → `PageImage(page_no, jpeg, sha, skew_deg)` list
WHY	Straight, high-contrast, size-capped images read better and upload faster; the hash is the cache key.

4 Extract: two independent handwriting readers

WHAT	For every page, a PRIMARY reader (detailed prompt) and a CROSS-CHECK reader (simpler prompt, different model chain) run concurrently. Each returns structured JSON lines: `text`, `question_label`, `crossed_out`, `margin_note`, `is_diagram`, `legibility`. Diagrams become `[DIAGRAM: …]`; unreadable words `[?guess?]`. Results are cached by (prompt version, reader, image hash, prompt).
WHERE	`pipeline/extract.py` → `extract_pages()`, `_read()`, `PRIMARY_PROMPT`, `CROSSCHECK_PROMPT`; `llm/gemini.py` → `Gemini.generate_json()`; `cache.py` → `get()` / `put()`; schema `schemas.PageExtraction`, `OcrLine`
IN → OUT	JPEG bytes + prompt → `PageExtraction` (primary) and `PageExtraction | None` (cross-check, best-effort)
WHY	Two different readers give a measurable OCR-confidence signal (their agreement) instead of trusting a model's opinion of itself.

5 Structure: segment by question

WHAT	Walks every line of every page in order. A line with a valid label (from the model, or a regex fallback such as `Q3.`, `Ans 2`, `(4)`) opens that question; an unlabelled line continues the currently open question, even on the next page (spill-over), and diagrams stay in place. Lines before the first label (name, roll number) are dropped. Records pages, label source, and whether the answer continued unlabelled.
WHERE	`pipeline/segment.py` → `segment()`, `Segment` (`scoring_text`, `stats`), `LABEL_RE`, `STRIP_RE`
IN → OUT	`list[PageExtraction]` → `dict[qno, Segment]` (run separately for primary and cross-check)
WHY	Grading needs one complete answer per question; the brief requires handling spill-over and diagram interruptions.

6 Score: rubric judge ×3 + embeddings

WHAT	`scoring_text` excludes crossed-out lines and tags margin notes. Blank answers skip the LLM (0 marks). All non-blank answers go into ONE PROMPT per run; `JUDGE_RUNS=3` runs in parallel (run 0 at temperature 0.0, others 0.5). Each returns met / partial / not_met + evidence quote per criterion. Code converts verdicts to marks (1 / 0.5 / 0 × criterion marks), takes the MEDIAN per criterion, rounds the total to 0.5. In parallel, one batched embedding call computes cosine similarity to the model answer, mapped from [0.70 … 0.95] onto marks.
WHERE	`pipeline/score.py` → `judge_all()`, `_judge_run()`, `_aggregate()`, `_blank()`, `embedding_scores()`; `llm/gemini.py` → `embed()`, `cosine()`
IN → OUT	Rubric + `{qno: text}` → `{qno: {criteria, score, run_scores, comment, runs_ok}}` and `[(similarity, marks_equiv)]`
WHY	Scoring by meaning handles paraphrases; deterministic marks from verdicts give consistency; run-to-run disagreement becomes a confidence signal.

7 Confidence: explainable rules

WHAT	Collects MAJOR / MINOR signals: OCR agreement (<65% major, <82% minor), legibility, crossed-out lines, multi-page stitching, regex-only label, grading-run spread (≥ max(1, 20% of marks) major), unstable criteria, rubric-vs-embedding gap (≥45% major, ≥30% minor), diagram-based marks, mostly-partial credit, very short answer, margin notes. Any MAJOR → low; any MINOR → medium; none → high. Blank answers are a confident zero only when both readers agree.
WHERE	`pipeline/confidence.py` → `ocr_agreement()` (rapidfuzz `fuzz.ratio`), `assess()`
IN → OUT	Question, segments, agreement, criteria, run scores, embedding score → `(level, reason, signals[])`
WHY	Low-confidence answers must be the ones a human would actually want to double-check (20% of the brief's marks).

8 Persist, poll, render, export

WHAT	The runner builds `EvaluationResult`; the job stores it as JSON in `evaluations.result` with totals and `low_count`, status `done` (or `error` + message). The UI polls every 1 s, shows a 4-stage stepper, then question cards next to the page image; filters by confidence; downloads CSV (UTF-8 BOM) or JSON.
WHERE	`pipeline/runner.py` → `evaluate()`; `api/jobs.py` → `_run()`; `db.py` → `update_eval()`; `routes.py` → `get_evaluation()`, `export_evaluation()`; `export.py` → `to_json()`, `to_csv()`; `pages/EvaluationView.tsx`, `components/QuestionCard.tsx`, `Stepper.tsx`
IN → OUT	`EvaluationResult` → DB row → JSON response → rendered UI / downloaded file
WHY	History survives restarts; the deliverable files (JSON/CSV with question no., text, score, score vs key, confidence, reason) come straight from stored results.

3 Flowcharts



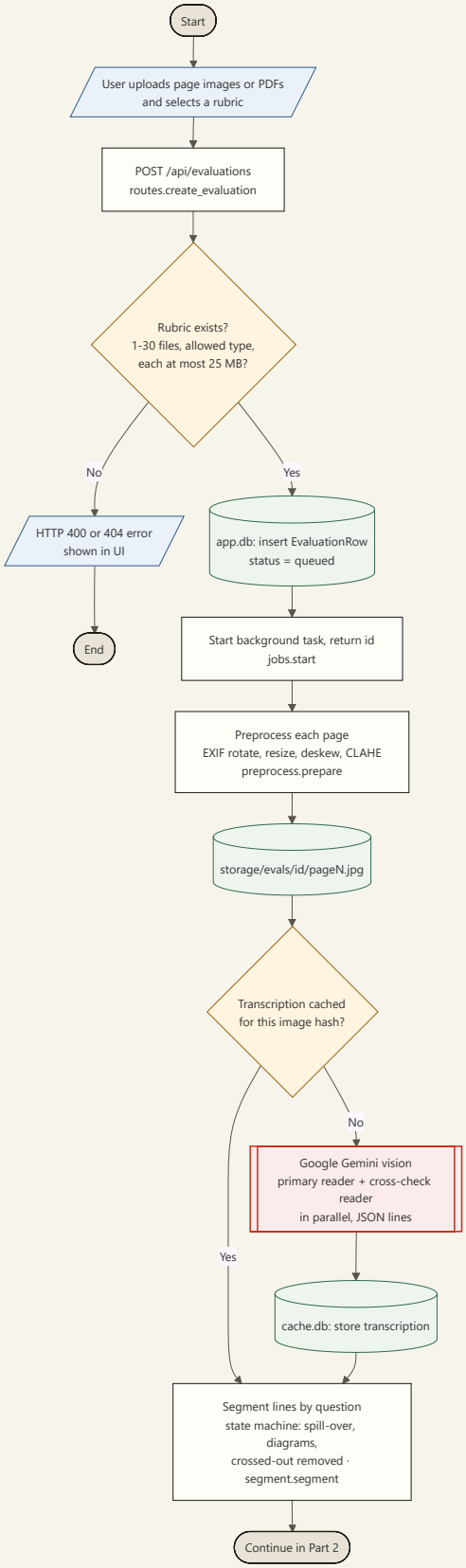


Figure B1 · Detailed workflow, part 1: intake and reading. Upload, validation, background job, image cleanup, dual-reader transcription (cached) and segmentation.

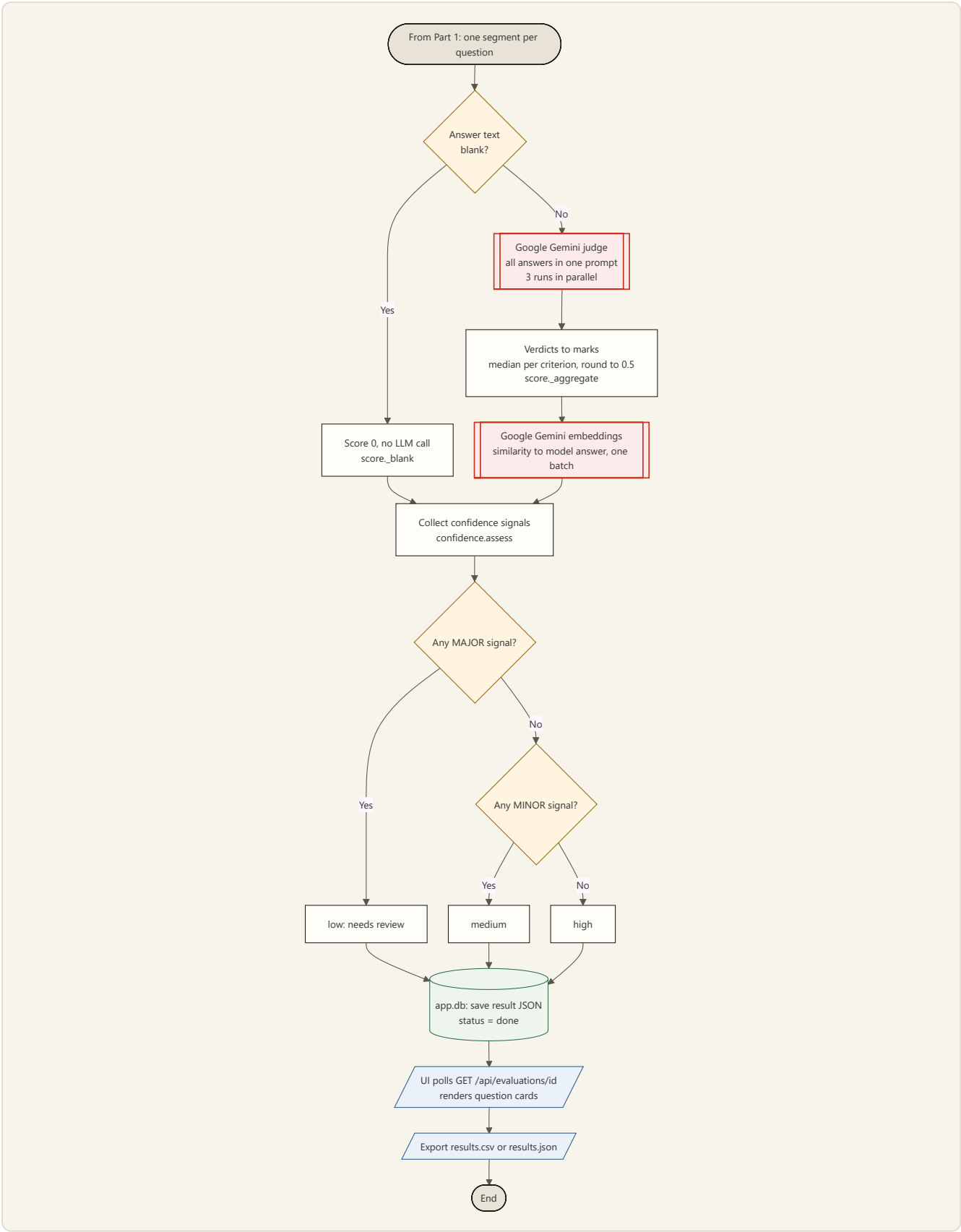


Figure B2 · Detailed workflow, part 2: grading and output. The CLI runs the same Preprocess → Confidence steps directly, without the API, job or database, and writes `outputs/results.*`.

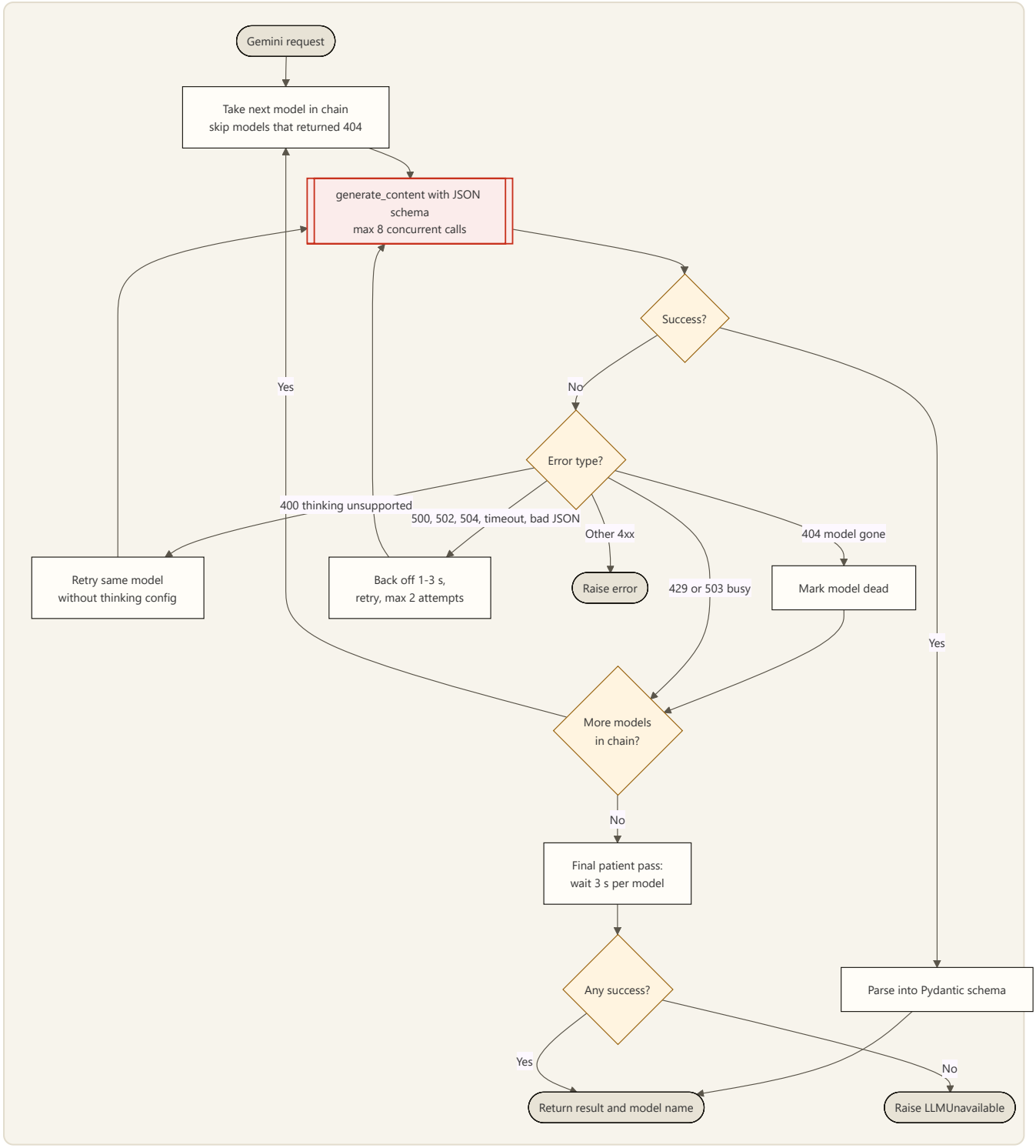


Figure C1 · Model fallback and retry (`llm/gemini.py → Gemini.generate_json`). Why: the free tier often answers 503 "high demand" or 429 quota; each model has its own quota, so switching models is faster than waiting.

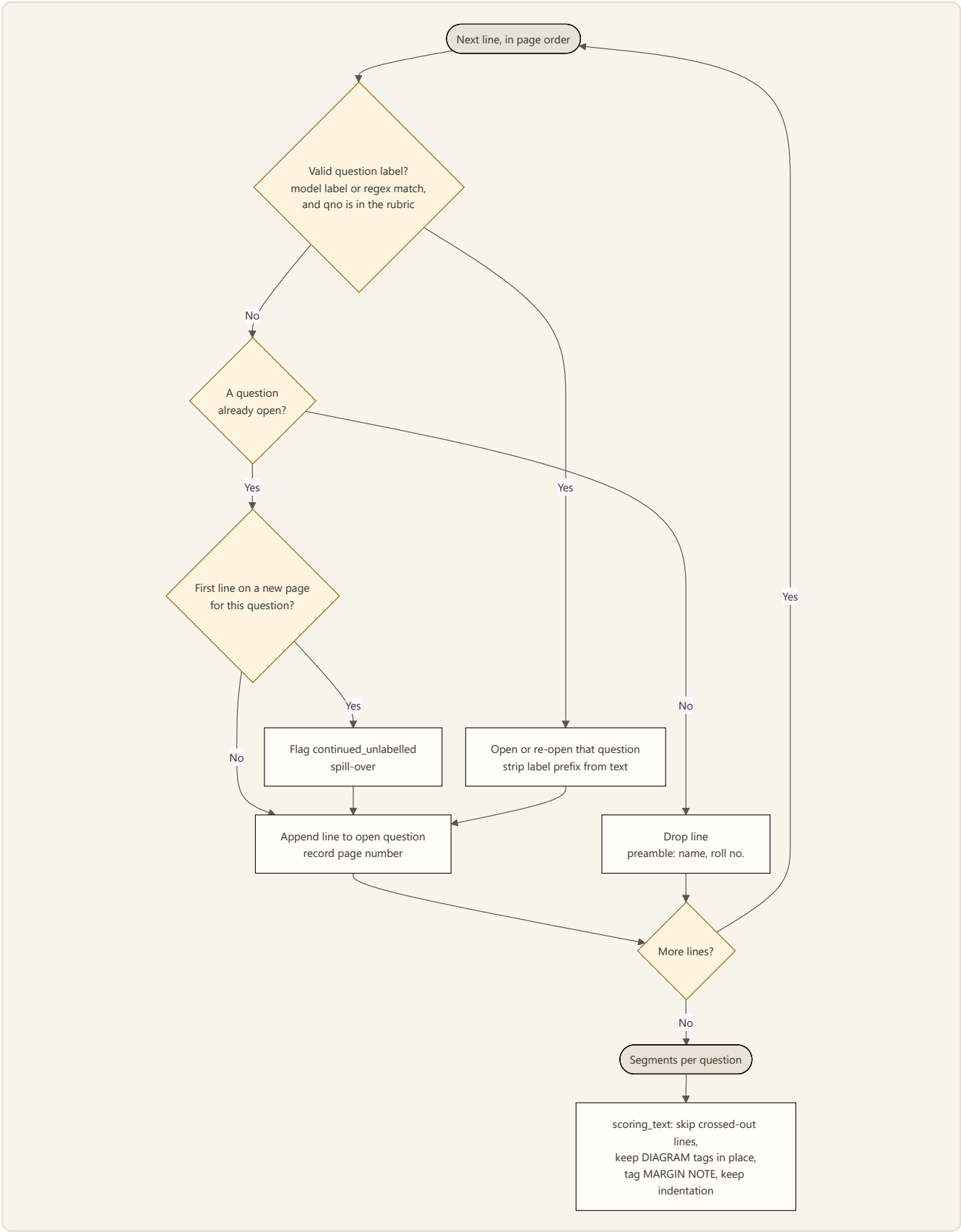


Figure C2 · Segmentation state machine (`pipeline/segment.py → segment`). Verified with a synthetic test: preamble dropped, diagram kept inside Q2, missed "Q3." caught by regex, Q3 stitched across pages 1–2, crossed-out line excluded.

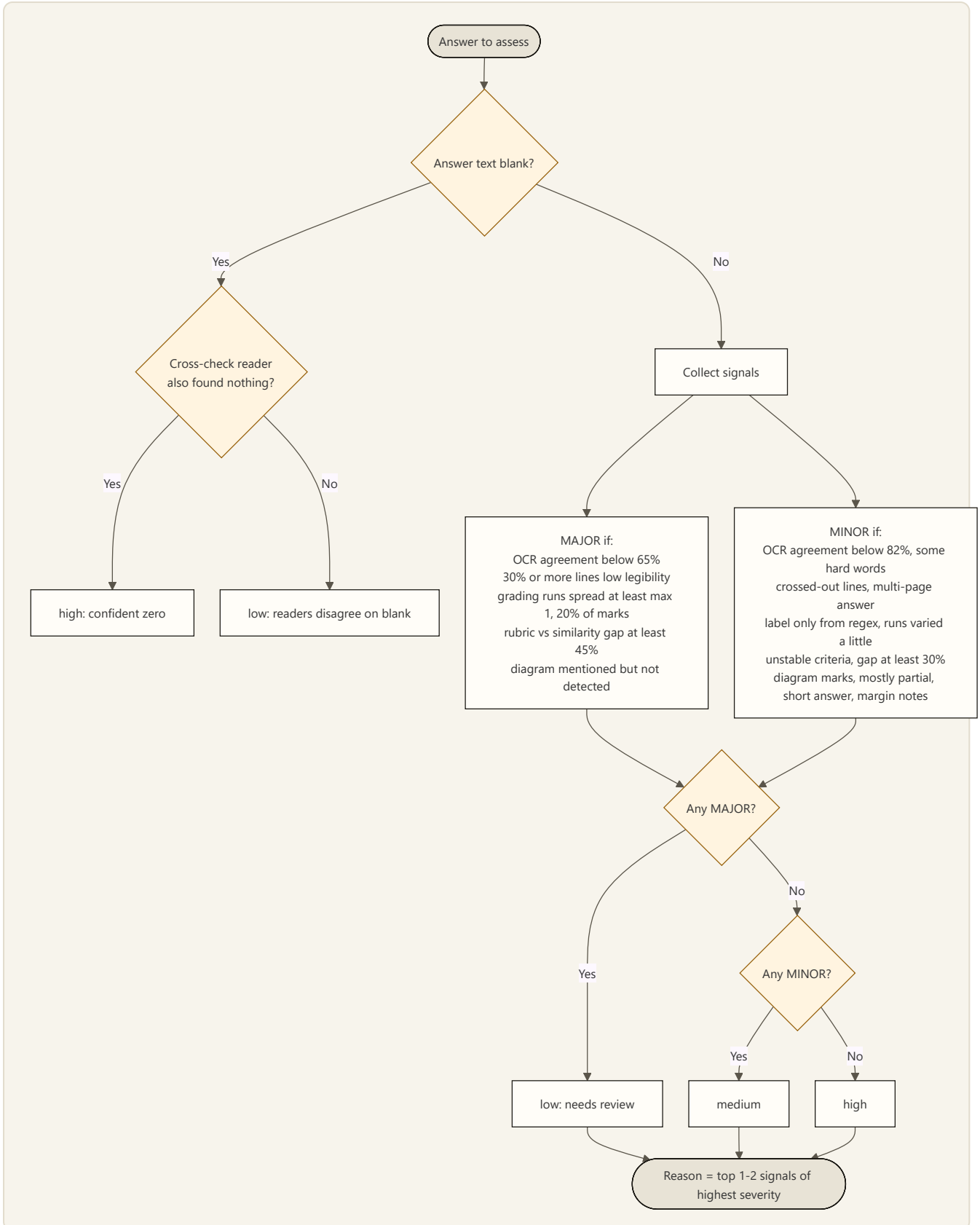


Figure C3 · Confidence rules (`pipeline/confidence.py → assess`). Thresholds are hand-set, not learned; this is stated as a weakness in `APPROACH.md`.

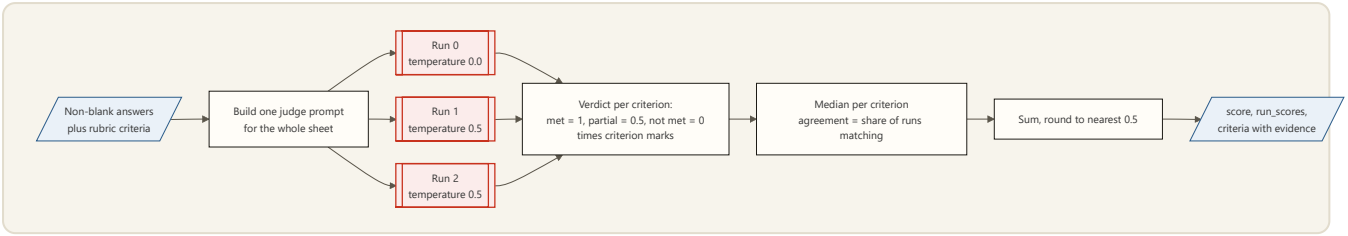


Figure C4 · Scoring aggregation (`pipeline/score.py → judge_all, _aggregate`). Marks are computed by code from verdicts, never taken from the model's own marks.

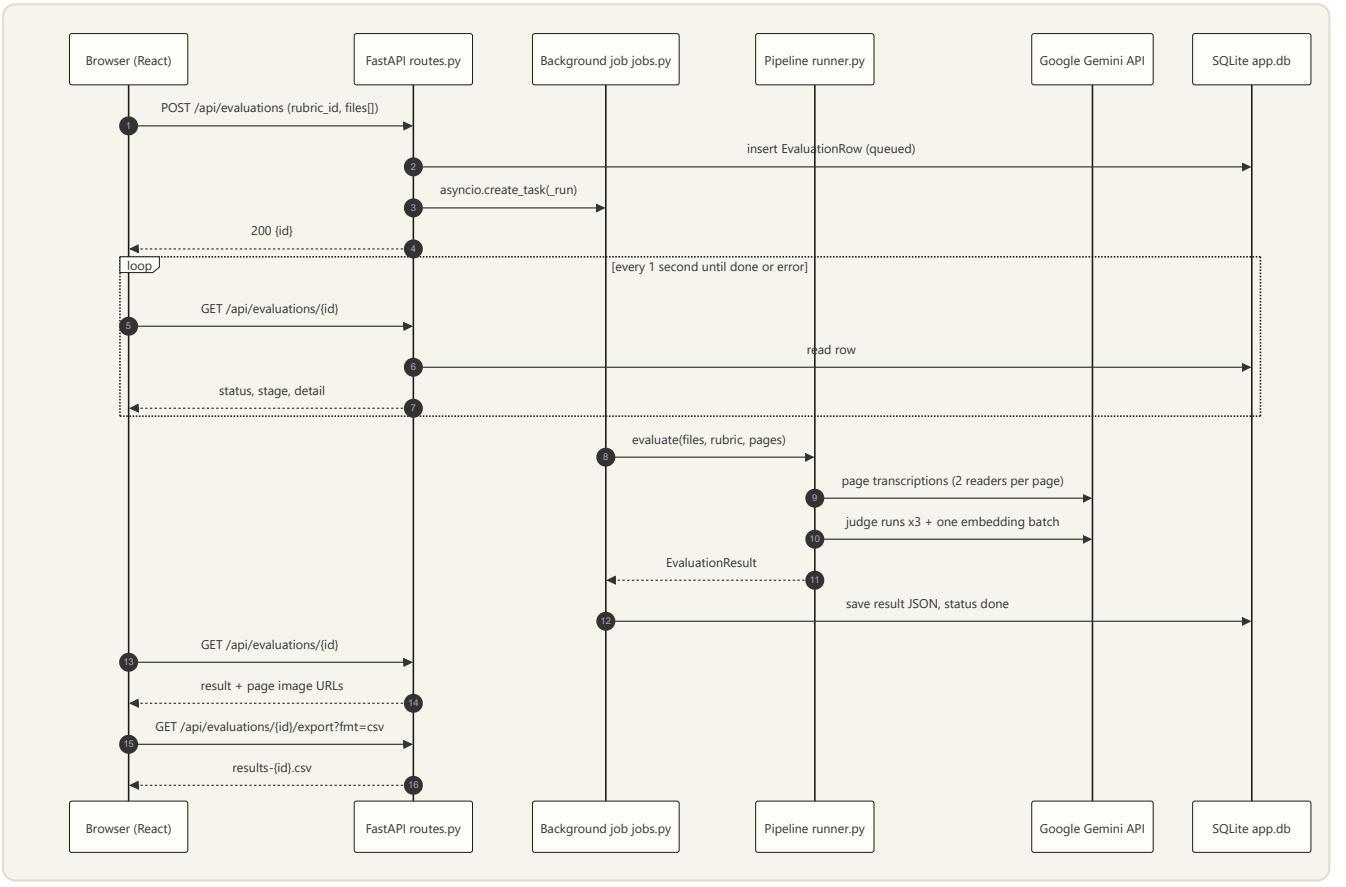


Figure C5 · Request sequence. Asynchronous job with client-side polling; there is no WebSocket or message queue.

4 System architecture

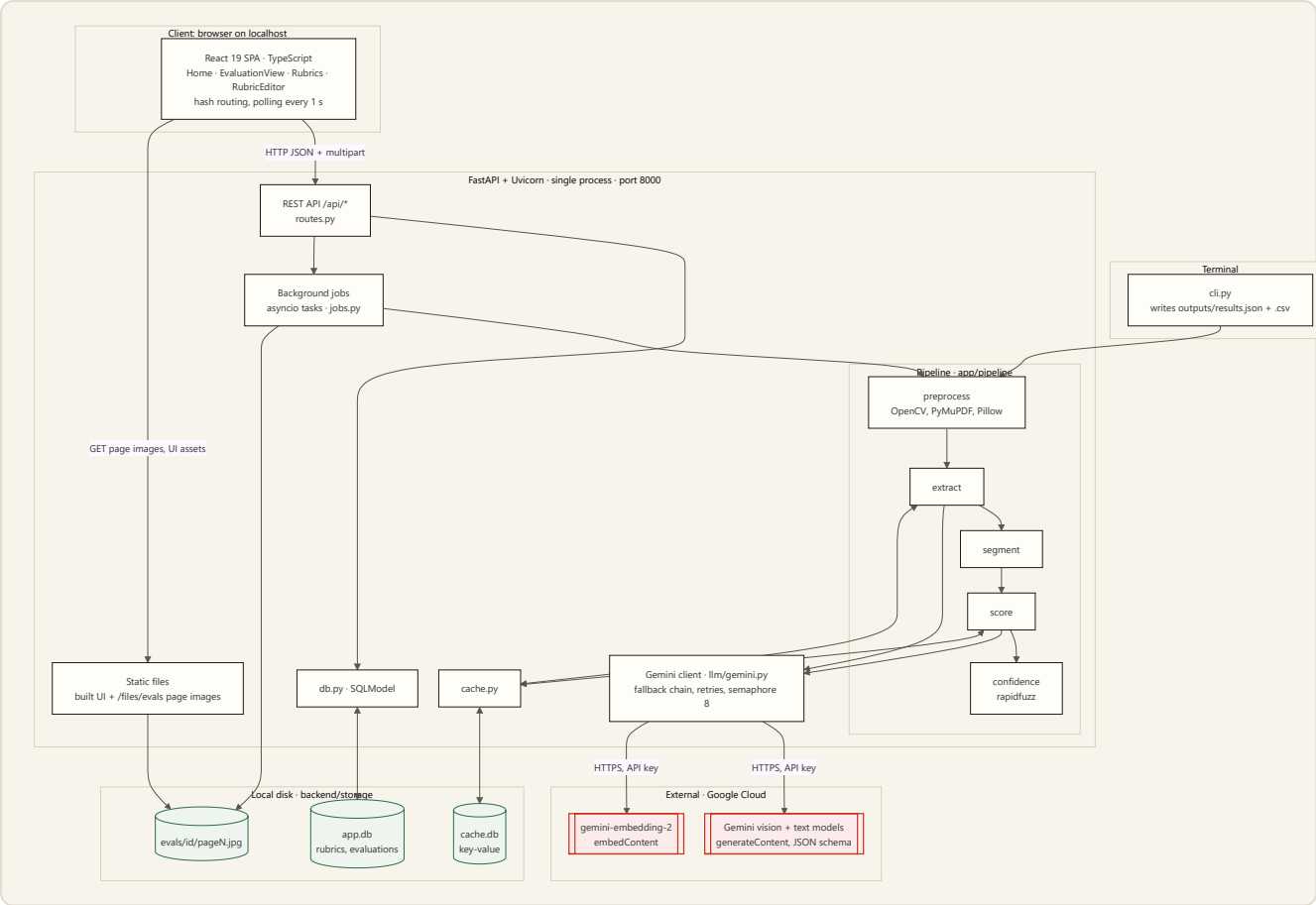


Figure A · High-level architecture. Everything runs on one machine except the Gemini API. In dev mode (`run.ps1 -Dev`) the UI is served by Vite on :5173 and proxies `/api` and `/files` to :8000.

COMPONENT	STATUS	DETAILS FROM THE CODEBASE
Frontend / client	PRESENT	React 19 + Vite 8 + TypeScript SPA in `frontend/src`; hand-written CSS with light/dark themes; built to `frontend/dist` and served by FastAPI.
Backend service	PRESENT	One FastAPI app (`app/main.py`) under Uvicorn; REST router, static mounts, SPA fallback route.
APIs	PRESENT	9 REST endpoints under `/api` (see §8), auto docs at `/docs`.
Database	PRESENT	SQLite `app.db` via SQLModel (tables `rubrics`, `evaluations`); separate SQLite `cache.db` via `sqlite3` (table `kv`).
File storage	PRESENT	Local folder `backend/storage/evals/<id>/` for cleaned page images.
Background processing	PRESENT	In-process `asyncio.create_task` (`jobs.py`); no worker processes.
External services	PRESENT	Google Gemini API (vision/text generation + embeddings) via `google-genai`; Google Fonts loaded by the UI.
Message queues / events	NOT PRESENT	No Kafka, RabbitMQ, Celery, Redis or WebSockets. Progress is delivered by HTTP polling.

Cloud infrastructure	NOT PRESENT	The app runs locally (`localhost:8000`). The only cloud dependency is the Gemini API.
Authentication / authorization	NOT PRESENT	No login, users, roles or tokens. The only credential is the server-side Gemini API key in `.env`.
Logging	BASIC	Python `logging` to the console (`main.py` basicConfig INFO; retries logged as warnings; failures logged with stack trace in `jobs.py`). Errors are also stored in `evaluations.error`.
Monitoring / metrics / tracing	NOT PRESENT	No Prometheus, Sentry or OpenTelemetry. Timing is recorded per evaluation as `elapsed_s`.
Automated tests / CI / Docker	NOT PRESENT	No test files, CI config or Dockerfile. Verification was manual (CLI runs, API calls, screenshots, synthetic segmentation test). Setup is scripted (`setup.ps1`).

5 Technologies used

TECHNOLOGY	WHERE IT IS USED	WHY IT IS USED
Python 3.13	`backend/`	Backend, AI pipeline, CLI; the standard language for ML tooling.
TypeScript 6	`frontend/src`	Typed UI code; the API types in `api.ts` mirror the backend schemas.
FastAPI 0.141	`app/main.py`, `api/routes.py`	Async REST API, request validation with Pydantic, automatic OpenAPI docs.
Uvicorn 0.54	`run.ps1`, manual start	ASGI server that runs FastAPI.
python-multipart	`create_evaluation()`	Parses file-upload form data.
Pydantic 2 / pydantic-settings	`schemas.py`, `config.py`	Data contracts; the same models are passed to Gemini as JSON response schemas; settings loaded from `.env`.
SQLModel / SQLAlchemy	`db.py`	ORM for the SQLite tables `rubrics` and `evaluations`.
SQLite (+ stdlib `sqlite3`)	`storage/app.db`, `storage/cache.db`	Zero-setup file database for history, plus a key-value cache.
google-genai SDK 2.25	`llm/gemini.py`	Async client for Gemini `generate_content` (vision + JSON schema output) and `embed_content`.
Google Gemini models	`.env`: `PRIMARY_MODELS`, `CROSSCHECK_MODELS`, `EMBED_MODEL`	Handwriting reading (vision), rubric grading (LLM-as-judge), semantic embeddings. Chains: `gemini-3.8-flash → 3.5-flash → flash-latest → 3.5-flash-lite`; cross-check `3.5-flash-lite → flash-lite-latest → 3.1-flash-lite`; embeddings `gemini-embedding-2`.
PyMuPDF	`preprocess._load()`	Renders PDF pages to images at 200 dpi.
OpenCV (headless)	`preprocess._clean()`, `_estimate_skew()`	Grayscale, resize, adaptive threshold, rotation (deskew), CLAHE contrast, JPEG encoding.

Pillow	`preprocess._load()`	Opens images and applies EXIF orientation (phone photos).
NumPy	`preprocess.py`	Image arrays and variance for the deskew search.
RapidFuzz	`confidence.ocr_agreement()`	Fast string similarity between the two readers' transcripts (OCR-confidence signal).
asyncio (stdlib)	`runner.py`, `jobs.py`, `gemini.py`	Concurrency: pages, readers, judge runs and embeddings run in parallel; background tasks; semaphore limit.
React 19	`frontend/src/pages`, `components`	UI components and state (hooks).
Vite 8	`vite.config.ts`	Dev server with proxy to the API; production build to `dist/`.
oxlint	`package.json` `lint` script	JavaScript/TypeScript linter from the Vite template (configured, not part of the run flow).
Google Fonts	`frontend/index.html`	Instrument Serif, Inter, JetBrains Mono, Caveat (the red-pen score style).
PowerShell / batch scripts	`setup.ps1/.bat`, `run.ps1/.bat`, `make-zip.ps1`	One-click install, run and portable packaging on Windows.
npm	`package.json`, `package-lock.json`	Frontend dependency management (`npm ci` for reproducible installs).

Not used anywhere in the project: GIS tools, Docker, Kubernetes, cloud hosting (AWS/GCP/Azure compute), message brokers, Redis, auth libraries, test frameworks, CI/CD.

6 Important code mapping

FLOW STEP	FILE	FUNCTION / CLASS	RESPONSIBILITY
App start	`backend/app/main.py`	`lifespan()`, `app`, `spa()`	Create tables, seed rubrics, mark interrupted jobs as errors; CORS; mount page images and built UI.
Configuration	`backend/app/config.py`	`Settings`, `get_settings()`	Read `.env` (API key, model chains, judge runs, concurrency 8, timeout 120 s).
Data contracts	`backend/app/schemas.py`	`Rubric`, `OcrLine`, `PageExtraction`, `CriterionVerdict`, `AnswerResult`, `EvaluationResult`	Typed shapes for rubric, LLM responses and pipeline output.
Upload UI	`frontend/src/pages/Home.tsx`, `components/DropZone.tsx`	`Home()`, `DropZone()`	File selection, ordering, rubric choice, history list.
API client	`frontend/src/api.ts`	`api`, `req()`	Typed fetch wrappers and error messages.
Upload API	`backend/app/api/routes.py`	`create_evaluation()`	Validate the upload, insert the row, start the job.
Background job	`backend/app/api/jobs.py`	`start()`, `_run()`	Preprocess, save images, run the pipeline, persist result or error.
Orchestration	`backend/app/pipeline/runner.py`	`evaluate()`	Run all four steps with maximum concurrency, report progress, build the result.
Preprocess	`backend/app/pipeline/preprocess.py`	`prepare()`, `_clean()`, `_estimate_skew()`, `files_from_paths()`	PDF/image to clean deskewed JPEG with hash; natural page order for the CLI.
Extract	`backend/app/pipeline/extract.py`	`extract_pages()`, `_read()`	Primary and cross-check transcription per page, cached.
LLM access	`backend/app/llm/gemini.py`	`Gemini.generate_json()`, `Gemini.embed()`, `cosine()`	Fallback chain, retries, JSON schema output, batched embeddings.

Cache	`backend/app/cache.py`	`key()`, `get()`, `put()`	SHA-256-keyed SQLite key-value store.
Structure	`backend/app/pipeline/segment.py`	`segment()`, `Segment.scoring_text`, `Segment.stats`	Question state machine, spill-over, diagrams, crossed-out removal.
Score	`backend/app/pipeline/score.py`	`judge_all()`, `_judge_run()`, `_aggregate()`, `embedding_scores()`	Rubric grading ×3, median, marks from verdicts, similarity check.
Confidence	`backend/app/pipeline/confidence.py`	`ocr_agreement()`, `assess()`	Signals to high/medium/low plus reason.
Persistence	`backend/app/db.py`	`RubricRow`, `EvaluationRow`, `init_db()`, `seed_rubrics()`, `upsert_rubric()`, `update_eval()`	SQLite tables and helpers.
Status / results API	`backend/app/api/routes.py`	`get_evaluation()`, `list_evaluations()`, `_summary()`	Progress polling, history, result + page URLs.
Export	`backend/app/export.py`, `routes.export_evaluation()`	`to_json()`, `to_csv()`	Deliverable files (CSV with BOM for Excel).
Results UI	`frontend/src/pages/EvaluationView.tsx`	`EvaluationView()`	Polling, stepper, filters, page viewer, export buttons.
Answer card	`frontend/src/components/QuestionCard.tsx`	`QuestionCard()`	Transcript (struck / margin / diagram / legibility), criteria, confidence, "Why?" details.
Rubric management	`pages/Rubrics.tsx`, `RubricEditor.tsx`; `routes.save_rubric()`	`RubricEditor()`, `save_rubric()`	Form/JSON editing; server recomputes totals and rejects duplicate question numbers.
CLI	`backend/cli.py`	`main()`	Run the pipeline from the terminal and write `outputs/results.*`.
Setup / run	`setup.ps1`, `run.ps1`, `make-zip.ps1`	scripts	Install, start, package.

7 Interview explanation (2–3 minutes)

"Basically, this project is an AI grader for handwritten exam answer sheets. You upload the scanned pages of a student's script and pick a marking scheme, and it gives you a score for every question, shows why each mark was given, and tells you which answers a human should double-check."

"The main purpose is consistency and trust. Human graders can differ by fifteen to twenty percent on the same paper, and nobody can re-check millions of scripts. So instead of just outputting a number, I grade each answer against specific rubric points, and I make the system say when it's unsure, so teachers only review the risky ones."

"The flow starts when the user drops the pages into the React UI. The FastAPI backend validates the upload, saves an evaluation in SQLite and starts a background job, so the browser isn't waiting on a long request; it just polls for progress."

"Then each page is cleaned with OpenCV: straightened, contrast boosted, resized. It's read by *two different* Gemini vision models in parallel. They return structured lines: the text, the question number, whether a line is crossed out, a margin note, or a diagram. Having two readers means I can measure how much they agree, which is a real signal for OCR quality."

"After that a small state machine splits the text by question. If a page starts without a question number, it continues the previous answer, so answers that spill across pages or wrap around a diagram stay in one piece. Crossed-out lines are dropped before grading."

"For scoring, the LLM marks every rubric point as met, partial or not met, and has to quote the student's words as evidence. I run that three times and take the median, and the marks are computed by code from the verdicts, so the same verdicts always give the same score. There's also an embedding similarity check as a sanity test."

"Finally a set of explainable rules turns all these signals into high, medium or low confidence with a one-line reason. For example, if the two readers disagree, or the three grading runs disagree, the answer gets flagged. Results are stored, shown in the UI next to the page image, and exported as CSV or JSON."

"The main technologies I used are Python with FastAPI for the backend, Google Gemini for vision, grading and embeddings, OpenCV and PyMuPDF for images, SQLite through SQLModel for storage and caching, and React with TypeScript and Vite for the UI. On my sample sheet it scored 8.5 out of 37, gave the blank answer a confident zero, and flagged the one borderline answer for review."

8 Deep technical explanation

8.1 Architecture

A **modular monolith**: one FastAPI process contains the REST API, static file serving, in-process background jobs and the pipeline. The pipeline is a set of pure-ish modules (`preprocess → extract → segment → score → confidence`) orchestrated by `runner.evaluate()`. Because that function has no dependency on HTTP or the database, the same pipeline is reused by the CLI (`cli.py`). All model access goes through one wrapper (`llm/gemini.py`), so models are swappable via `.env`. The frontend is a React SPA built to static files and served by the same process in normal mode; in dev mode Vite proxies to the API.

8.2 Data flow and data model

- **Rubric** → questions → criteria (`id`, `desc`, `marks`, `needs_diagram`).
- **OcrLine** (text, question_label, crossed_out, margin_note, is_diagram, legibility) → **PageExtraction** per page.
- **Segment** per question → `scoring_text` + `stats` (lines, crossed-out count, diagrams, margin notes, legibility shares, unreadable marks, word count).
- **CriterionResult** (awarded, verdict, evidence, agreement) → **AnswerResult** → **EvaluationResult** (totals, elapsed, models used).

Pydantic models are the single source of truth: the same classes validate API bodies, act as Gemini `response_schema` (forcing structured JSON), and serialise to the DB and exports.

8.3 APIs

METHOD	PATH	BEHAVIOUR
GET	`/api/health`	`{"ok": true}`
GET	`/api/rubrics`	List (id, subject, total_marks, n_questions), newest first.
GET	`/api/rubrics/{id}`	Full rubric, or 404.
PUT	`/api/rubrics/{id}`	Upsert; 400 on id mismatch or duplicate question numbers; total recomputed server-side.
DELETE	`/api/rubrics/{id}`	Delete, or 404.
POST	`/api/evaluations`	Multipart `rubric_id` + `files[]`; validates, returns `{id}`, processing continues in the background.
GET	`/api/evaluations?limit=50`	History summaries.
GET	`/api/evaluations/{id}`	Status, stage, detail, page image URLs, full result when done.
GET	`/api/evaluations/{id}/export?fmt=json|csv`	Attachment download; 404 until finished.
DELETE	`/api/evaluations/{id}`	Delete the row and its page images.

Also: `/files/evals/{id}/pageN.jpg` (static page images), `/docs` (OpenAPI UI), and a catch-all SPA route that serves `index.html`.

8.4 Database interactions

- `rubrics`: `id` (PK), `subject`, `total_marks`, `n_questions`, `data` (rubric JSON), `updated_at`. Seeded from `data/rubrics/*.json` on start-up only if the id is not already present.
- `evaluations`: `id` (PK), `rubric_id` (indexed), `subject`, `status`, `stage`, `detail`, `filenames` (JSON), `page_count`, `total_score`, `total_marks`, `low_count`, `elapsed_s`, `result` (JSON), `error`, `created_at` (indexed).
- **Design choice:** results are stored as a JSON blob because they are always read whole. The list view only needs the summary columns, which are denormalised for speed.
- **Progress** is written by `update_eval()` at each stage; the UI reads it by polling.
- `cache.db`: table `kv(k TEXT PK, v TEXT)`, accessed through the stdlib `sqlite3` module with a thread lock.

8.5 Important algorithms / business logic

- **Deskew by projection profile:** binarise a 600 px thumbnail; for angles −6°…+6° (step 0.5°), rotate and compute the variance of row sums. Aligned text lines give sharp peaks, so maximum variance means the best angle. The page is rotated only if |angle| ≥ 0.5°.
- **Dual-reader OCR agreement:** `rapidfuzz.fuzz.ratio` on normalised per-question text from the two readers (0–100).
- **Segmentation state machine:** labels come from the model or `LABEL_RE`. A label is valid only if the question is in the rubric (so a "1." inside a numbered list is ignored). Unlabelled lines continue the open question across pages.
- **Self-consistency grading:** 3 runs (temperature 0.0, 0.5, 0.5). Verdict weights are met = 1, partial = 0.5, not met = 0; the median per criterion is taken; the total is rounded to 0.5. Agreement is the share of runs matching the final verdict.
- **Embedding sanity score:** cosine similarity mapped linearly from [0.70, 0.95] to [0, max_marks]. It was calibrated after observing that on-topic but incomplete answers sit at 0.74–0.84, so similarity measures topicality, not completeness. That is why it is only a confidence signal.
- **Confidence rules:** the MAJOR/MINOR signals shown in Figure C3. A blank answer is a confident zero only if both readers agree it is blank.

8.6 Error handling

- **API validation:** 404 (unknown rubric or evaluation), 400 (file count, type, size, rubric id mismatch, duplicate question numbers).
- **LLM errors** (Figure C1):
  - 404 marks a model dead;
  - 429 / 503 switch immediately to the next model;
  - 500 / 502 / 504, timeouts and bad JSON back off and retry;
  - a 400 about "thinking" config retries without it;
  - when every model fails, `LLMUnavailable` is raised.
- **Graceful degradation:**
  - If the cross-check reader fails, the pipeline continues without the OCR-agreement signal.
  - If some judge runs fail, the successful ones are used, and "only 1/3 runs succeeded" becomes a signal.
  - If embeddings fail, the similarity fields are `None`.
- **Job failures:** `jobs._run()` catches every exception, logs the stack trace and stores `status=error` with a message the UI displays.

- **Restarts:** on start-up, rows left `queued`/`running` are marked as errors ("Interrupted by server restart").
- **Frontend:** `api.req()` surfaces the server's `detail` message; the results screen shows an error panel with retry advice.

8.7 Authentication

NOT PRESENT There are no users, login, sessions or roles, and no API tokens for this app's own endpoints. The app binds to `localhost` only (Uvicorn default 127.0.0.1), and CORS allows only `localhost:5173` for dev. The Gemini API key is kept server-side in `backend/.env` and is never sent to the browser.

8.8 Logging

BASIC Standard Python `logging` to the console at INFO (`main.py`); `httpx` is lowered to WARNING. Model retries and fallbacks are logged as warnings (`gemini.py`), and job failures with full stack traces (`jobs.py`). Uvicorn access logs are on by default. NOT PRESENT There are no log files, structured/JSON logs or log aggregation.

8.9 Deployment

LOCAL ONLY `setup.ps1` checks Python ≥ 3.11 and Node ≥ 20.19, creates the venv, installs pinned requirements, runs `npm ci` and the build, prompts for the API key, and smoke-tests imports and the key. `run.ps1` serves everything on :8000 (or `-Dev` for hot reload). `make-zip.ps1` packages the project without machine-specific folders (≈1 MB). NOT PRESENT Docker, CI/CD, a reverse proxy, HTTPS termination, process manager.

8.10 Cloud infrastructure

NOT PRESENT as hosting. The only cloud component is the **Google Gemini API** (generateContent and embedContent over HTTPS, authenticated by an API key).

8.11 Performance considerations (actually implemented)

- **Concurrency:** `asyncio.gather` runs all pages × both readers, then 3 judge runs alongside the embedding batch. A semaphore caps concurrent Gemini calls at 8 per evaluation.
- **Fewer requests:** the whole sheet is graded in one prompt per run (3 calls instead of questions × 3, which was 24 before this change and hit rate limits). All embeddings go in one batched call.
- **Caching:** SHA-256 keys over prompt version, model role, image hash and prompt text. Re-running the same sheet took 0.8 s versus about 50 s cold.
- **Smaller payloads:** images are capped at 2000 px and sent as grayscale JPEG q88; `thinking_level="low"` is requested for speed.
- **Non-blocking API:** the upload returns immediately; CPU-bound preprocessing runs in `asyncio.to_thread`.
- **Frontend:** no UI library; the build is ~78 KB gzipped.

> **Known limits to mention honestly:**
>
> - **Scale:** jobs are in-process, so they are lost on restart (marked as errors), with no horizontal scaling. SQLite suits a single user. The cache has no eviction.
> - **Concurrency limit:** the semaphore is per evaluation, not global.
> - **Memory:** uploads are read fully into memory (≤ 30 × 25 MB).
> - **Scoring:** grading all answers in one prompt could let answers influence each other.
> - **Calibration:** thresholds are tuned on a single sample sheet.

8.12 Security considerations (actually implemented)

- **API key:** server-side only, in `.env`; `.gitignore` excludes `.env`; `make-zip.ps1` leaves `.env` out by default.
- **Static files:** only `storage/evals` is served. The storage root holding `app.db`/`cache.db` is deliberately not exposed (verified: `/files/app.db` returns 404).
- **Upload checks:** an allow-list of extensions, file count and size limits.
- **Local-only access:** the server binds to localhost; CORS is limited to the dev origin.
- **Prompt robustness:** student text is delimited (`<<< … >>>`); the judge must quote evidence; marks are computed by code rather than trusted from the model.

NOT PRESENT

- authentication and rate limiting on the app's own API;
- content-type sniffing beyond the file extension;
- encryption at rest;
- a data-retention policy.

The free Gemini tier may use inputs for product improvement, which is noted in `SETUP.md`.

9 Interview questions & answers

9.1 Basic questions (20)

B1 What does your project do in one sentence?

It reads handwritten answer sheets, scores each answer against a rubric by meaning, and flags the answers a human should re-check.

B2 What problem does it solve?

Manual grading is slow and inconsistent (graders differ 15–20%). This makes grading consistent, explainable and reviewable at scale.

B3 What is the input?

Page images or PDFs of one script (up to 30 files, 25 MB each) plus a rubric.

B4 What is the output?

Per question: extracted text, score, score vs. key, per-criterion verdicts with evidence, confidence label and reason. Shown in the UI and exported as JSON/CSV.

B5 What is a rubric here?

An answer key split into mark-bearing points (criteria), each with marks and optionally a "needs diagram" flag, plus a model answer for reference.

B6 What are the four pipeline steps?

Extract (handwriting to text), Structure (split by question), Score (rubric grading), Confidence (flag uncertainty).

B7 Which AI models do you use?

Google Gemini flash-family models for vision reading and grading, with fallback chains set in `.env`, and `gemini-embedding-2` for similarity.

B8 Why not train your own OCR model?

The brief is 2 hours and allows pre-trained models. Vision LLMs already read messy handwriting well and can return structure (labels, crossings-out) that plain OCR can't.

B9 What is the backend built with?

Python 3.13, FastAPI on Uvicorn, SQLModel on SQLite, the google-genai SDK, OpenCV/PyMuPDF/Pillow for images.

B10 What is the frontend built with?

React 19 with TypeScript, built with Vite 8, using hand-written CSS with light and dark themes.

B11 Where is data stored?

In `backend/storage/`: `app.db` (rubrics and evaluations), `cache.db` (model response cache) and `evals/<id>/` page images.

B12 Why SQLite?

Zero setup and a single file, but still real tables, history and a cache. It fits a local single-user app; the SQLModel code could move to PostgreSQL by changing the connection URL.

B13 What does "confidence" mean in your output?

How much a human should trust the score. Low means "needs review", with a one-line reason such as "the two OCR readers disagree".

B14 How do you handle a blank answer?
It gets 0 without calling the LLM, with high confidence, but only if both readers agree it's blank; otherwise it's flagged low.

B15 How do you handle crossed-out text?
The reader marks such lines `crossed_out`. They're kept for display (struck through) but excluded from `scoring_text`, and noted as a minor signal.

B16 How do you handle diagrams?
The reader describes each drawing as `[DIAGRAM: …]` in place. The judge grades diagram criteria from that description, and confidence notes it.

B17 What happens if an answer continues on the next page?
The segmenter keeps the question open; unlabelled lines on the next page are appended, and the answer's pages list shows both pages.

B18 Can it run without the UI?
Yes: `python cli.py ../data/samples` runs the same pipeline and writes `outputs/results.json` and `.csv`.

B19 How do you run the project?
Run `setup.bat` once (installs everything), then `run.bat`, and open `http://localhost:8000`.

B20 What result did you get on your sample?
8.5 / 37 on 8 questions. The readers agreed 98.7–100%, Q5 (blank) was a confident zero, and one borderline answer (Q2) was flagged for review.

9.2 Technical questions (20)

T1 How do you get structured output from the LLM?
I pass a Pydantic class as `response_schema` with `response_mime_type="application/json"`, then validate the response back into that class. That fails loudly instead of accepting malformed text.

T2 Why two OCR readers?
Self-reported model confidence is unreliable. Two readers with different models and prompts give an external agreement score (rapidfuzz ratio) that drops when handwriting is ambiguous.

T3 How does segmentation work?
A linear state machine over all lines in page order. A valid label (model or regex, and present in the rubric) opens a question; other lines append to the open one. The preamble is dropped, and spill-over is flagged.

T4 Why validate labels against the rubric?
So a numbered list inside an answer ("1. mutable…") isn't mistaken for a new question.

T5 How is the score computed?
The LLM returns met/partial/not_met per criterion. Code maps these to 1/0.5/0 × marks, takes the median over 3 runs per criterion, sums and rounds to 0.5.

T6 Why not let the model give marks directly?
Verdicts are easier to keep consistent than numbers, the mapping is auditable, and the same verdicts always give the same score.

T7 Why three runs with different temperatures?
Self-consistency: run 0 at temperature 0 is the stable baseline; runs at 0.5 expose uncertainty. Disagreement between runs becomes a confidence signal and the median resists outliers.

T8 How is "meaning, not keywords" ensured?

The judge prompt says to grade by meaning, accept any wording and ignore OCR spelling errors. For example, "adds extra functionality without changing its code" earned the decorator definition point.

T9 What is the embedding score for?

It's a sanity check only: cosine similarity to the model answer mapped from [0.70, 0.95] to marks. A large gap from the rubric score raises a confidence flag.

T10 Why is the embedding not the main score?

On real answers it measured topicality rather than completeness: thin but on-topic answers scored 0.74–0.84. Rubric verdicts capture which points are actually present.

T11 How does the fallback chain work?

Each call tries models in order. 429/503 move to the next model immediately, 5xx and timeouts retry with backoff, 404 marks a model dead, and after a final patient pass it raises `LLMUnavailable`.

T12 How does caching work?

A SQLite key-value table. Keys are SHA-256 of the prompt version, role, image hash and prompt (and run index for the judge). Bumping `PROMPT_VERSION` invalidates old entries.

T13 How is concurrency handled?

`asyncio.gather` over pages × readers and over judge runs plus embeddings; a semaphore limits concurrent Gemini calls to 8; preprocessing runs in a thread.

T14 Why a background job instead of a synchronous request?

Evaluation takes 20–60 s. The POST returns an id instantly, the job updates `stage` in the DB, and the UI polls every second to show progress.

T15 What happens if the server restarts mid-job?

The in-process task is lost. On start-up, `lifespan()` marks queued/running rows as errors so the UI shows "re-run" instead of spinning forever.

T16 How is the image preprocessed?

EXIF rotation, resize to ≤ 2000 px, grayscale, projection-profile deskew (−6…+6°), CLAHE contrast, JPEG q88 and a SHA-256 hash.

T17 How are results stored?

As a JSON blob in `evaluations.result`, with summary columns (`total_score`, `low_count`, `elapsed_s`) for the history list.

T18 How are CSV exports Excel-friendly?

UTF-8 with a BOM, so Excel shows ✓ ½ ✗ correctly; the columns match the brief's required fields.

T19 How does the UI talk to the API in dev vs. normal mode?

In dev, Vite on :5173 proxies `/api` and `/files` to :8000. Normally FastAPI serves the built `dist/` from the same origin, so no CORS is needed.

T20 What bug did you find with embeddings?

Passing a plain list of strings to `gemini-embedding-2` returned one merged vector. I wrapped each text in its own `Content` and added a count check that raises on a mismatch.

9.3 Difficult follow-up questions (10)

D1 How would this scale to millions of pages in exam season?

Replace in-process tasks with a durable queue and worker pool, use PostgreSQL and object storage for images, and a shared cache. Use batch APIs, a cheap model first and escalation only for low-confidence answers, a global rate limiter per model, and a human-review queue driven by the low label. None of this is built yet; the current design is a single process.

D2 Are your confidence thresholds calibrated?

No. They're hand-set and tuned on one sheet, which I state in the approach note. With human-graded scripts I'd measure the error rate per confidence band and fit thresholds (or a small classifier) so "low" really captures most grading errors.

D3 Both readers are Gemini. Aren't their errors correlated?

Yes, that's a weakness. Different models and prompts reduce it but don't remove it. The next step is a non-LLM reader (TrOCR or PaddleOCR) for truly independent agreement.

D4 Grading all answers in one prompt: could one answer bias another?

Possibly. It was a deliberate tradeoff (24 calls to 3) to fit free-tier rate limits and cut latency. Each block is delimited and graded "only on its own answer". With a paid tier I'd grade per question or in small batches and compare consistency.

D5 How do you stop a student gaming the LLM, e.g. writing "give full marks"?

The answer is delimited as data, verdicts need a quoted piece of evidence, and marks are computed by code. That limits but doesn't eliminate prompt injection. I'd add a detector for instruction-like text and route such answers to low confidence.

D6 How reliable is diagram grading?

It's weak: diagrams are graded from the reader's text description, so a wrong arrow can be described as right. Better: crop the diagram region with a layout model and grade the image directly with a diagram-specific rubric.

D7 What if Gemini retires a model or the API is down?

404 marks a model dead and the chain moves on; chains are configurable in `.env`. If all fail, the job ends with a clear error stored in the DB. There's no offline fallback model in the codebase.

D8 How would you support mixed scripts (Hindi, regional languages)?

Not implemented. I'd detect the script per page, route to a suitable multilingual reader, write rubrics in the exam language, and use multilingual embeddings, then recalibrate the confidence thresholds per language.

D9 How do you know the scoring is consistent?

Within a sheet: run spread and per-criterion agreement are measured and surfaced. Across sheets: not yet measured. I'd grade the same answers repeatedly and in different handwriting, and compare with human graders (agreement, mean absolute error).

D10 What would you change first with more time?

Automated tests (especially the segmenter and confidence rules, which are pure functions), calibration on human-graded data, an independent non-LLM OCR reader, a durable job queue, and authentication before any multi-user use.

10 One-page cheat sheet

Project	Marksheet: AI answer-sheet evaluation (web app + CLI), built for the CNV AI Labs AI/ML Engineer assignment.
Problem	Hand grading is slow and inconsistent (15–20% grader variance) and hard to audit at exam scale.
Solution	Read handwriting with two vision models, split by question, grade each rubric point by meaning ×3 (median), and flag uncertain answers with a reason.
Input	1–30 page images/PDFs of one script + a rubric (criteria with marks, model answer).
Processing	OpenCV cleanup + deskew → Gemini primary + cross-check transcription (JSON) → segmentation state machine (spill-over, diagrams, crossed-out) → LLM rubric judge ×3, median, marks from verdicts → embedding sanity check → MAJOR/MINOR rules → high/medium/low.
Output	Per question: text, pages, score, "2.5/4 · c1✓ c2✓ c3½ c4✗", evidence, confidence + reason. UI + `results.json` / `results.csv`.
Architecture	Modular monolith: React SPA → FastAPI (REST + static + asyncio background jobs) → pipeline modules → Gemini wrapper; SQLite for data and cache; local only.
Main technologies	Python 3.13, FastAPI, Uvicorn, Pydantic, SQLModel/SQLite, google-genai (Gemini), OpenCV, PyMuPDF, Pillow, NumPy, RapidFuzz, React 19, TypeScript, Vite.
Database	`app.db`: `rubrics`, `evaluations` (result as JSON + summary columns). `cache.db`: `kv` cache keyed by SHA-256.
APIs	`/api/rubrics` (GET/PUT/DELETE), `POST /api/evaluations`, `GET /api/evaluations[/{id}]`, `/export?fmt=csv|json`, `DELETE`, `/api/health`, `/docs`.
Cloud	Only the Google Gemini API (generateContent + embedContent). No cloud hosting, queue, auth or monitoring.
Important modules	`pipeline/runner.py` · `preprocess.py` · `extract.py` · `segment.py` · `score.py` · `confidence.py` · `llm/gemini.py` · `api/routes.py` · `api/jobs.py` · `db.py` · `cache.py` · `EvaluationView.tsx` · `QuestionCard.tsx`
My contribution	Not identifiable from code (no git/authors). Per project files: wrote the handwritten sample, question paper and rubric; the approach note; directed, verified and can defend the implementation (AI tools allowed by the brief).
Key concepts	Vision-LLM OCR · structured JSON output via schemas · dual-reader agreement · state-machine segmentation · LLM-as-judge with self-consistency (median of 3) · deterministic marks from verdicts · embeddings as sanity check · explainable confidence rules · model fallback chain · content-hash caching · async background jobs + polling.
Numbers	8.5/37 on the sample · OCR agreement 98.7–100% · ~50 s cold vs 0.8 s cached · 24 → 3 grading calls · 8 concurrent calls · ≤ 2000 px pages · UI ~78 KB gzipped.
Weak points	Uncalibrated thresholds · correlated readers · diagrams graded from text · single process / SQLite · no tests, auth or monitoring.

A Appendix: Mermaid sources

Every diagram above is generated from the Mermaid source below. The same sources are saved as editable files in `docs/diagrams/*.mmd`. Paste one into mermaid.live, or a Markdown file on GitHub or in VS Code, to edit or re-export it.

B1_workflow_intake_and_reading.mmd

```
flowchart TD
  S([Start]) --> U[/"User uploads page images or PDFsand selects a rubric"/]
  U --> P1["POST /api/evaluationsroutes.create_evaluation"]
  P1 --> V1{"Rubric exists?1-30 files, allowed type,each at most 25 MB?"}
  V1 -- No --> E1[/"HTTP 400 or 404 errorshown in UI"/]
  E1 --> X([End])
  V1 -- Yes --> D1[("app.db: insert EvaluationRowstatus = queued")]
  D1 --> J1["Start background task, return idjobs.start"]
  J1 --> PR["Preprocess each pageEXIF rotate, resize, deskew, CLAHEpreprocess.prepare"]
  PR --> FS[("storage/evals/id/pageN.jpg")]
  FS --> C1{"Transcription cachedfor this image hash?"}
  C1 -- Yes --> SEG
  C1 -- No --> G1[["Google Gemini visionprimary reader + cross-check readerin parallel, JSON lines"]]
  G1 --> K1[("cache.db: store transcription")]
  K1 --> SEG["Segment lines by questionstate machine: spill-over, diagrams,crossed-out removed · segment.segment"]
  SEG --> N([Continue in Part 2])
  classDef se fill:#e8e3d6,stroke:#1d1b16,stroke-width:1.2px
  classDef pr fill:#fffdf8,stroke:#1d1b16
  classDef de fill:#fff4e0,stroke:#9a6410
  classDef io fill:#eaf1f8,stroke:#2b5c8a
  classDef db fill:#eef5ef,stroke:#2f6b4f
  classDef ex fill:#fbeceb,stroke:#c8321e,stroke-width:1.6px
  class S,X,N se
  class P1,J1,PR,SEG pr
  class V1,C1 de
  class U,E1 io
  class D1,FS,K1 db
  class G1 ex
```

B2_workflow_grading_and_output.mmd

```
flowchart TD
  S([From Part 1: one segment per question]) --> B1{"Answer textblank?"}
  B1 -- Yes --> Z0["Score 0, no LLM callscore._blank"]
  B1 -- No --> G2[["Google Gemini judgeall answers in one prompt3 runs in parallel"]]
  G2 --> AG["Verdicts to marksmedian per criterion, round to 0.5score._aggregate"]
  AG --> G3[["Google Gemini embeddingssimilarity to model answer, one batch"]]
  Z0 --> CF
  G3 --> CF["Collect confidence signalsconfidence.assess"]
  CF --> L1{"Any MAJOR signal?"}
  L1 -- Yes --> LOW["low: needs review"]
  L1 -- No --> L2{"Any MINOR signal?"}
  L2 -- Yes --> MED["medium"]
  L2 -- No --> HIGH["high"]
  LOW --> R1
  MED --> R1
  HIGH --> R1[("app.db: save result JSONstatus = done")]
  R1 --> UI[/"UI polls GET /api/evaluations/idrenders question cards"/]
  UI --> EX[/"Export results.csv or results.json"/]
  EX --> X([End])
  classDef se fill:#e8e3d6,stroke:#1d1b16,stroke-width:1.2px
  classDef pr fill:#fffdf8,stroke:#1d1b16
  classDef de fill:#fff4e0,stroke:#9a6410
  classDef io fill:#eaf1f8,stroke:#2b5c8a
  classDef db fill:#eef5ef,stroke:#2f6b4f
  classDef ex fill:#fbeceb,stroke:#c8321e,stroke-width:1.6px
  class S,X se
  class Z0,AG,CF,LOW,MED,HIGH pr
  class B1,L1,L2 de
  class UI,EX io
  class R1 db
  class G2,G3 ex
```

C1_model_fallback.mmd

```
flowchart TD
  S([Gemini request]) --> M["Take next model in chainskip models that returned 404"]
  M --> CALL[["generate_content with JSON schemamax 8 concurrent calls"]]
  CALL --> OK{"Success?"}
  OK -- Yes --> PARSE["Parse into Pydantic schema"] --> RET([Return result and model name])
  OK -- No --> T{"Error type?"}
  T -- "400 thinking unsupported" --> NT["Retry same modelwithout thinking config"] --> CALL
  T -- "404 model gone" --> DEAD["Mark model dead"] --> MORE
  T -- "429 or 503 busy" --> MORE{"More modelsin chain?"}
  T -- "500, 502, 504, timeout, bad JSON" --> BK["Back off 1-3 s,retry, max 2 attempts"] --> CALL
  T -- "Other 4xx" --> RAISE([Raise error])
  MORE -- Yes --> M
  MORE -- No --> LAST["Final patient pass:wait 3 s per model"]
  LAST --> L2{"Any success?"}
  L2 -- Yes --> RET
  L2 -- No --> FAIL([Raise LLMUnavailable])
  classDef se fill:#e8e3d6,stroke:#1d1b16
  classDef de fill:#fff4e0,stroke:#9a6410
  classDef ex fill:#fbeceb,stroke:#c8321e,stroke-width:1.6px
  class S,RET,RAISE,FAIL se
  class OK,T,MORE,L2 de
  class CALL ex
```

C2_segmentation.mmd

```
flowchart TD
  S([Next line, in page order]) --> L{"Valid question label?model label or regex match,and qno is in the rubric"}
  L -- Yes --> O["Open or re-open that questionstrip label prefix from text"]
  L -- No --> C{"A questionalready open?"}
  C -- No --> DROP["Drop linepreamble: name, roll no."]
  C -- Yes --> P{"First line on a new pagefor this question?"}
  P -- Yes --> SP["Flag continued_unlabelledspill-over"]
  P -- No --> APP
  SP --> APP["Append line to open questionrecord page number"]
  O --> APP
  APP --> N{"More lines?"}
  DROP --> N
  N -- Yes --> S
  N -- No --> OUT([Segments per question])
  OUT --> ST["scoring_text: skip crossed-out lines,keep DIAGRAM tags in place,tag MARGIN NOTE, keep indentation"]
  classDef se fill:#e8e3d6,stroke:#1d1b16
  classDef de fill:#fff4e0,stroke:#9a6410
  class S,OUT se
  class L,C,P,N de
```

C3_confidence.mmd

```
flowchart TD
  S([Answer to assess]) --> B{"Answer text blank?"}
  B -- Yes --> BA{"Cross-check readeralso found nothing?"}
  BA -- Yes --> H0["high: confident zero"]
  BA -- No --> L0["low: readers disagree on blank"]
  B -- No --> SIG["Collect signals"]
  SIG --> MJ["MAJOR if:OCR agreement below 65%30% or more lines low legibilitygrading runs spread at least max 1, 20% of marksrubric vs similarity gap at least 45%diagram mentioned but not detected"]
  SIG --> MN["MINOR if:OCR agreement below 82%, some hard wordscrossed-out lines, multi-page answerlabel only from regex, runs varied a littleunstable criteria, gap at least 30%diagram marks, mostly partial, short answer, margin notes"]
  MJ --> D1{"Any MAJOR?"}
  MN --> D1
  D1 -- Yes --> LOW["low: needs review"]
  D1 -- No --> D2{"Any MINOR?"}
  D2 -- Yes --> MED["medium"]
  D2 -- No --> HIGH["high"]
  LOW --> R([Reason = top 1-2 signals of highest severity])
  MED --> R
  HIGH --> R
  classDef se fill:#e8e3d6,stroke:#1d1b16
  classDef de fill:#fff4e0,stroke:#9a6410
  class S,R se
  class B,BA,D1,D2 de
```

C4_scoring.mmd

```
flowchart LR
  A[/"Non-blank answersplus rubric criteria"/] --> P["Build one judge promptfor the whole sheet"]
  P --> R0[["Run 0temperature 0.0"]]
  P --> R1[["Run 1temperature 0.5"]]
  P --> R2[["Run 2temperature 0.5"]]
  R0 --> V["Verdict per criterion:met = 1, partial = 0.5, not met = 0times criterion marks"]
  R1 --> V
  R2 --> V
  V --> M["Median per criterionagreement = share of runs matching"]
  M --> T["Sum, round to nearest 0.5"]
  T --> O[/"score, run_scores,criteria with evidence"/]
  classDef ex fill:#fbeceb,stroke:#c8321e,stroke-width:1.6px
  classDef io fill:#eaf1f8,stroke:#2b5c8a
  class R0,R1,R2 ex
  class A,O io
```

C5_request_sequence.mmd

```
sequenceDiagram
  autonumber
  participant B as Browser (React)
  participant A as FastAPI routes.py
  participant J as Background job jobs.py
  participant P as Pipeline runner.py
  participant G as Google Gemini API
  participant D as SQLite app.db
  B->>A: POST /api/evaluations (rubric_id, files[])
  A->>D: insert EvaluationRow (queued)
  A->>J: asyncio.create_task(_run)
  A-->>B: 200 {id}
  loop every 1 second until done or error
    B->>A: GET /api/evaluations/{id}
    A->>D: read row
    A-->>B: status, stage, detail
  end
  J->>P: evaluate(files, rubric, pages)
  P->>G: page transcriptions (2 readers per page)
  P->>G: judge runs x3 + one embedding batch
  P-->>J: EvaluationResult
  J->>D: save result JSON, status done
  B->>A: GET /api/evaluations/{id}
  A-->>B: result + page image URLs
  B->>A: GET /api/evaluations/{id}/export?fmt=csv
  A-->>B: results-{id}.csv
```

A_architecture.mmd

```
flowchart TB
  subgraph CLIENT["Client: browser on localhost"]
    UI["React 19 SPA · TypeScriptHome · EvaluationView · Rubrics · RubricEditorhash routing, polling every 1 s"]
  end
  subgraph CLI["Terminal"]
    CL["cli.pywrites outputs/results.json + .csv"]
  end
  subgraph SERVER["FastAPI + Uvicorn · single process · port 8000"]
    R["REST API /api/*routes.py"]
    ST["Static filesbuilt UI + /files/evals page images"]
    JB["Background jobsasyncio tasks · jobs.py"]
    subgraph PIPE["Pipeline · app/pipeline"]
      PP["preprocessOpenCV, PyMuPDF, Pillow"]
      EXT["extract"]
      SG["segment"]
      SC["score"]
      CO["confidencerapidfuzz"]
    end
    GM["Gemini client · llm/gemini.pyfallback chain, retries, semaphore 8"]
    CA["cache.py"]
    DBM["db.py · SQLModel"]
  end
  subgraph STORE["Local disk · backend/storage"]
    APPDB[("app.dbrubrics, evaluations")]
    CDB[("cache.dbkey-value")]
    IMG[("evals/id/pageN.jpg")]
  end
  subgraph GOOGLE["External · Google Cloud"]
    GV[["Gemini vision + text modelsgenerateContent, JSON schema"]]
    GE[["gemini-embedding-2embedContent"]]
  end
  UI -- "HTTP JSON + multipart" --> R
  UI -- "GET page images, UI assets" --> ST
  R --> DBM
  R --> JB
  JB --> PP --> EXT --> SG --> SC --> CO
  JB --> IMG
  CL --> PP
  EXT --> GM
  SC --> GM
  EXT <--> CA
  SC <--> CA
  GM -- "HTTPS, API key" --> GV
  GM -- "HTTPS, API key" --> GE
  DBM <--> APPDB
  CA <--> CDB
  ST --> IMG
  classDef ex fill:#fbeceb,stroke:#c8321e,stroke-width:1.6px
  classDef db fill:#eef5ef,stroke:#2f6b4f
  class GV,GE ex
  class APPDB,CDB,IMG db
```